# Ejemplo Mínmo de uso

Contiene el código minimo necesario para iniciar una escena con:
- Un protagonista (sin cuerpo) que maneja la cámara con wasd y el ratón.
- Paredes de ladrillo sobre los bordes de la escena.
- Un actor con el que se puede interactuar al presionar “E” mientras se le apunta con el cursor.
- Una flor y un árbol cargados como utileria.

## 1. Descripción General

El archivo `main.js` actúa como el punto de entrada principal (**entry point**) de una aplicación construida con el motor Threeatro.

Demuestra la integración completa del sistema:
1. Definición de manifiestos de recursos (imágenes de la interfaz, texturas para bloques y modelos 3D en formato GLTF).
2. Procesamiento asíncrono de recursos a través de la clase `Tramoyista` [cite: 15].
3. Creación de clases personalizadas extendiendo entidades base ( `Actor` y `Escena` )[cite: 15].
4. Generación procedimental de límites en la grilla del mundo de voxeles[cite: 15].
5. Colocación de utilería y configuración del DOM de la interfaz de usuario ( `ui` )[cite: 15].
6. Secuencia de arranque del `Director` una vez cargado el DOM mediante `window.onload` [cite: 15].

## 2. Definición de Manifiestos de Recursos

El script organiza los **assets** del proyecto mediante arreglos de objetos clave-valor antes de enviarlos al cargador[cite: 15]:

### 2.1 Herramientas e Ítems ( `items` )

Contiene las imágenes que componen la barra inferior o inventario del jugador[cite: 15]:

```
const items = [
    { nombre: 'espada', ruta: './assets/pico.png' },
    { nombre: 'pico', ruta: './assets/pico.png' },
    { nombre: 'pico', ruta: './assets/pico.png' }
];
```

### 2.2 Texturas del Mundo ( `texturas` )

Mapea imágenes para el recubrimiento visual de la matriz de voxeles del `Mundo` [cite: 15]. La posición del elemento en el arreglo (más uno) define el ID numérico utilizado en `mundo.colocar()` o `mundo.crear_suelo()` [cite: 15]:
- **ID 1**: `"cesped"` ( `./assets/cesped.png` )[cite: 15]
- **ID 2**: `"piedra"` ( `./assets/piedra.png` )[cite: 15]
- **ID 3**: `"arena"` ( `./assets/arena.png` )[cite: 15]
- **ID 4**: `"tablon"` ( `./assets/tablon.png` )[cite: 15]
- **ID 5**: `"ladrillo"` ( `./assets/ladrillo.png` )[cite: 15]
- **ID 6**: `"tronco"` ( `./assets/tronco.png` )[cite: 15]
- **ID 7**: `"agua"` ( `./assets/agua.png` )[cite: 15]

### 2.3 Modelos Tridimensionales ( `modelos` )

Define los archivos de malla y geometría en formato GLTF que pueden ser instanciados en el mundo[cite: 15]:

```
const modelos = [
    { nombre: 'arbol', ruta: './assets/arbol3D.gltf' },
    { nombre: 'flor', ruta: './assets/modelos3D/flor.gltf' },
    { nombre: 'personajeA', ruta: './assets/modelos3D/personaje_prueba.gltf' }
];
```

---

# 3. Declaración de Clases Personalizadas

---

### 3.1 Clase `Juanito` (Extiende `Actor` )

Representa un personaje no jugable (**NPC**) interactivo con lógica propia[cite: 15]:

| Método | Descripción |
|---|---|
| `constructor()` | Invoca `super('Juanito', 'personajeA', 8, 2, 5, [], 0.05)` para establecer el nombre, modelo GLTF asignado, posición inicial (8, 2, 5) y factor de escala 0.05[cite: 15]. |
| `iniciar()` | Hook de inicialización llamado al cargar la escena (disponible para setups internos)[cite: 15]. |
| `actualizar()` | Hook ejecutado en cada fotograma del bucle principal (**render loop**) [cite: 15]. |
| `al_interactuar()` | Se dispara automáticamente cuando el protagonista apunta con el cursor al personaje y presiona la tecla `E` [cite: 15]. Ejecuta `this.decir('Soy juanito.')` para mostrar la caja de texto[cite: 15]. |
| `cada_segundo()` | Hook ejecutado con frecuencia de 1 segundo (ej. para patrullaje o lógica de movimiento gradual)[cite: 15]. |

### 3.2 Clase `Pruebas` (Extiende `Escena` )

Define la estructura del mapa, las entidades iniciales y la interfaz en pantalla[cite: 15]:

#### Construcción del Mundo Voxel

En el método `iniciar()` , la escena dibuja una sala delimitada por paredes utilizando el API del `Mundo` [cite: 15]:

1. **Piso**: `this.mundo.crear_suelo(1, 1)` coloca bloques de césped (ID: 1) a la altura Y=1[cite: 15].
2. **Paredes**: Cuatro llamadas a `this.mundo.poner_seccion(2, ...)` colocan bloques de piedra (ID: 2) formando un perímetro cerrado de altura Y=1 a Y=7 en una grilla de $20x20$[cite: 15].
3. **Sincronización**: Se ejecuta `this.mundo.actualizar()` para solicitar al motor la construcción gráfica de las geometrías de los bloques[cite: 15].

#### Instanciación de Objetos y Actores

- **Utilería (Árbol)**: `this.crear_utileria("arbol 1", "arbol", 5, 1, 5, 0.1)` coloca el modelo 3D en (5, 1, 5) escalado a 0.1[cite: 15].

- **Utilería (Flor con Rotación)**: `this.crear_utileria("tulipan", "flor", 7, 2, 7, 0.01, [90, 0, 0])` inserta el objeto con una inclinación de 90 grados en el eje X[cite: 15].
- **Actor**: `this.agregar_actor(Juanito)` instancia y añade la clase del NPC a la colección activa de la escena[cite: 15].

### Configuración de la Interfaz ( `UI` )

Inyecta marcado HTML directamente en las esquinas de la pantalla mediante los contenedores del motor[cite: 15]:

- `this.ui.superior_izquierdo` : Muestra estadísticas como dinero y cantidad de semillas[cite: 15].
- `this.ui.superior_derecho` : Muestra el reloj e indicador de días del juego[cite: 15].

---

# 4. Ciclo de Inicialización de la Aplicación

La ejecución asíncrona se gestiona dentro del evento `window.onload` [cite: 15]:

```
window.onload = async function () {
    // 1. Cargar y procesar los assets con el Tramoyista
    const recursos = await tramoyista.generar_catalogo_de_recursos();

    // 2. Instanciar el Director pasando la base de datos de recursos
    const director = new Director(recursos);

    // 3. Crear la escena inicial pasándole el Director
    const pruebas = new Pruebas(director);

    // 4. Precargar recursos globales en el motor y asignar la escena
    director.precargar();
    director.escena_actual = pruebas;

    // 5. Iniciar el bucle de renderizado y lógica del juego
    director.iniciar();
};
```